Insertion Sort -

Algorithm (Step-by-step):

- Start from the 2nd element (index 1).
- Compare it with elements before it.
- Shift larger elements one position ahead.
- Insert the current element into its correct position.
- Repeat until the entire array is sorted.

Time Complexity:

- Best case: $O(n)$
- Worst case: $O(n^2)$
- Average case: $O(n^2)$
- Space Complexity: $O(1)$ (in-place sorting)

Insertion Sort -

Ques. Sort an array using Insertion Sort.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        # Shift elements greater than key
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

arr = [12, 11, 13, 5, 6]
print("Sorted Array:", insertion_sort(arr))
```

Merge Sort -

Algorithm (Step-by-step):

- Divide → Split the array into two halves.
- Conquer → Recursively sort each half.
- Combine → Merge the two sorted halves into one sorted array.

Time Complexity:

- Best case: O(n log n)
- Worst case: O(n log n)
- Average case: O(n log n)
- Space Complexity: O(n) (extra space for merging)

Key Point: Very stable & efficient; used in external sorting (large data).

Merge Sort -

Ques. Sort an array using Merge Sort.

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left_half = arr[:mid]
        right_half = arr[mid:]
        # Recursive call
        merge_sort(left_half)
        merge_sort(right_half)
        i = j = k = 0
        # Merge two halves
        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1
        # Copy remaining elements
        while i < len(left_half):
            arr[k] = left_half[i]
            i += 1
            k += 1
        while j < len(right_half):
            arr[k] = right_half[j]
            j += 1
            k += 1
    return arr
# Example
arr = [38, 27, 43, 3, 9, 82, 10]
print("Sorted Array:", merge_sort(arr))
```

Merge Sort -

Ques. Sort an array using Merge Sort.

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left_half = arr[:mid]
        right_half = arr[mid:]
        # Recursive call
        merge_sort(left_half)
        merge_sort(right_half)
        i = j = k = 0

# Example
arr = [38, 27, 43, 3, 9, 82, 10]
print("Sorted Array:", merge_sort(arr))
```

Merge Sort -

Ques. Sort an array using Merge Sort.

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left_half = arr[:mid]
        right_half = arr[mid:]
        # Recursive call
        merge_sort(left_half)
        merge_sort(right_half)
        i = j = k = 0
         # Merge two halves
          while i < len(left_half) and j < len(right_half):
              if left_half[i] < right_half[j]:
                  arr[k] = left_half[i]
                  i += 1
              else:
                  arr[k] = right_half[j]
                  j += 1
              k += 1
```

Merge Sort -

Ques. Sort an array using Merge Sort.

```
# Merge two halves
    while i < len(left_half) and j < len(right_half):
        if left_half[i] < right_half[j]:
            arr[k] = left_half[i]
            i += 1
        else:
            arr[k] = right_half[j]
            j += 1
        k += 1
    # Copy remaining elements
    while i < len(left_half):
        arr[k] = left_half[i]
        i += 1
        k += 1
    while j < len(right_half):
        arr[k] = right_half[j]
        j += 1
        k += 1
return arr
```